

DB Demo Studio — 动态演示预览区交互解决方案设计文档

面向 AI Agent 与前端架构师的交互模式与技术实现规范

文档版本: V1.0 目标系统: 数据库协作式教学演示工作台 设计策略: 兼顾科技感、多端响应与高并发健壮性

一、引言与背景概述

在 DB Demo Studio 系统中，“一切皆对话”是核心理念，而右侧的**动态演示预览区 (Demo Preview)** 则是课堂教学和演示的最终舞台。预览区必须能够承载高密度的可视化资产（包含 6 阶段即时演示、Mermaid 图表、D3.js 动画配置以及双会话并发事务状态）。

为了解决前述分析中提及的“三栏布局响应式拥挤”与“大图展示空间不足”的痛点，本设计文档针对**方案一（三栏全景内联联动板）**和**方案二（对话流卡片触发式弹窗/抽屉）**均给出了完整的界面设计、交互流转及技术降级实现方案，并最终提供了一套兼收并蓄的**混合式模态架构**。

二、方案一：三栏全景内联画布方案 (Inline Panorama Panel)

1. 界面布局与视觉呈现

该方案坚持传统的左-中-右三栏平铺布局，右侧演示区作为一个**持久存在 (Always-on)** 的巨型独立画布视窗。

- 画布容器**：采用深色背景 (#0f172a)，与中间的灰色聊天流形成视觉对比。利用高对比度的网格背景线 (Grid Overlay) 强化科技感与精准度。
- 三视图联动 Tab 轴**：顶部采用扁平化、毛玻璃质感的选项卡，控制 FlowEditor、ExecutionPlayer 和 Animation Engine 的切换。底色带有细微的蓝光呼吸灯动效，提示当前视图的状态。
- 全局时间线 (Timeline Slider)**：位于视窗底部，提供一个横向覆盖全宽的播放轴。刻度清晰标记当前步骤（例如 Step 3: Query Optimize），并有高亮指针指示教师的物理进度。

2. 核心交互流转与控制逻辑

任何在中间对话流 (Chat Panel) 中发生的变更，无需用户点击确认，都会实时、静默地通过前端状态总线同步至右侧。

- 当 AI 在对话流中输出 [v3] JOIN 查询演示 快照时，右侧画布启动骨架屏加载，渲染引擎（如 Mermaid 渲染器）在后台解析生成的文本。
- 解析完成后，画布运用平滑的过渡动画 (CSS Transition / D3 easeCubic) 将旧版本的执行计划树替换为新版本。

3. **多模态联动点击**：教师在右侧画布中直接点击执行计划中的某一个算子（如 Hash Join），中间的输入框会自动填充对应的探查指令（例如“解释为什么这里选择 Hash Join 而非 Nested Loop”），实现双向反馈。

✂ 方案一专用优化：三栏响应式自适应策略（折叠机制）

由于三栏并排在小屏幕（如 13-14 寸笔记本）上会导致极度拥挤，本方案引入了“**弹性折叠与侧拉（Flex-Drawer-Collapse）**”技术。

当浏览器视窗宽度缩减至低于 1280px 时，系统自动将左侧 ConversationPanel 折叠为窄条图标栏（仅保留状态标签与头像）。同时，在中间与右侧边界交界处提供一个**隐形热区双击条（Splitter-Bar）**。教师可通过快捷键 Alt + M 或双击该条，使中间对话流完全向左收缩，右侧演示画布瞬间扩增至 85% 宽度，从而为复杂的 B+ 树或大型 ER 图争取到充足的展示空间。

三、方案二：对话流卡片触发式弹窗/抽屉方案 (Card-Triggered Modal)

1. 界面布局与视觉呈现

该方案倡导“**轻量、聚焦、按需显示**”的极简主义哲学。默认情况下，页面仅呈现左侧列表与中间对话流（双栏布局），右侧不占用任何固定物理空间。

- **对话卡片（Card Asset）**：AI 生成的演示成果在对话流中表现为一个极具设计感的“多维数据卡片”（包含缩略图、技术指标、版本标签和明显的 [点击全屏探查] 按钮）。
- **模态叠 layers（Overlay Modal）**：用户点击后，一个覆盖全屏、带有高斯模糊（Backdrop Filter: blur(12px)）的深色全屏浮层自右向左滑出。
- **分屏对比视窗**：弹窗内部天然支持“左/右双视窗并排”或“上/下对比”，专门为 P2 阶段的事务隔离级别模拟器设计，可完美模拟会话 A 与会话 B 锁冲突的时间轴线。

2. 核心交互流转与控制逻辑

此方案的交互心智是“**深度专注模式（Deep Focus Mode）**”。

1. AI 推送演示快照卡片，卡片右上角闪烁微型绿光指示器，代表“已在后台就绪”。
2. 教师点击卡片，触发全屏弹窗。此时，教学视线完全聚焦于可视化动画，完全隔离聊天流的杂讯干扰。
3. 弹窗内右上角常驻一个微型“流式对话挂件（Floating Chat Widget）”，允许教师在全屏看图时，依然能输入诸如“插入键值 42”的指令，弹窗内动画会根据 WebSocket 返回的数据即时刷新。
4. 点击空白处或按下 ESC 键，弹窗优雅淡出，视线切回对话主流程。

✂ 方案二专用优化：多会话状态留存与多层模态挂载隔离

弹窗方案最令人头疼的是关闭后状态丢失。本方案在 React 前端引入了“**虚拟多终端影子节点 (Shadow-DOM Instance Mounter)**”技术。

每个对话卡片生成的演示，其 D3 渲染上下文并没有挂载在全局 Window 上，而是序列化存储在本地前端的 Shadow Memory Canvas Manager 中。这意味着教师即使关闭了当前的 B+ 树演示弹窗，去跟 AI 聊了三句别的知识点，再度点击该卡片时，弹窗能在一毫秒内恢复到关闭前的精确动画帧与节点分裂状态，彻底解决了频繁开关带来的重新加载感。

四、 两套交互方案的多维度对比矩阵

为了客观评估两套方案在真实教学和技术实现中的表现，以下从多个核心业务与技术指标进行量化对比：

评估指标	方案一：三栏全景内联画布方案	方案二：对话卡片触发式弹窗方案
屏幕空间利用率	较低 长期挤占 40% 的宽度，小屏设备表现不佳。	极高 默认不占空间，点击后独占 100% 全屏，大图展示极佳。
认知负荷与心智模式	连贯同步 “所见即所得”，左侧聊天与右侧画面实时对应，符合整体感知。	存在断层 弹窗的弹出与关闭破坏了对话流的连续性，需要切换思维。
多端与响应式适配	极高风险 需要编写大量侧边栏折叠、动态拖拽热区的复杂 JS 代码。	天然适配 弹窗基于 Viewport 宽高，对平板、窄屏笔记本十分友好。
高并发与同步延迟	较高负载 每次流式微调都需要持续重绘右侧，长连接数据量巨大。	低频高效 仅在弹窗激活时维持高频重绘，后台卡片仅接收结构化 JSON。
典型适用教学场景	P0 即时 6 阶段讲解、实时交互式样式调色（如“改成蓝色系”）。	P2 事务双会话隔离对比、复杂的 B+ 树多节点分裂动画追踪。

五、 终极演进：混合式双模态架构解决方案 (Hybrid Dual-Mode Layout)

单一的选择非黑即白，鉴于 **DB Demo Studio** 面临的场景既包含轻量级的 P0/P1 图表调优（需要实时连贯），又包含复杂的 P2 模拟器演示（需要极致空间），我们提出最终的 **混合式双模态交互设计 (Hybrid Dual-Mode Architecture)**：

💡 混合方案核心设计：“内联锚定 + 一键悬浮剧场”

1. **默认模态（常规联动）**：系统默认采用三栏平铺布局。当 AI 生成 P0 6 阶段讲解或轻量级 Mermaid 流程图时，直接在右侧内联画布中展现。此时教师能一边改 Prompt，一边看图变换，维持良好的交互反馈链路。
2. **剧场最大化按钮（Theater Mode Toggle）**：在右侧内联视窗的右上角，常驻一个醒目的 [🎬 剧场专注/全屏] 交互微标。教师一旦发现当前的 ER 图过于庞大，或正在讲解 P2 级别的并发锁冲突，点击该按钮，右侧画布立刻脱离标准的 DOM 三栏流，以 `position: fixed` 全屏模态弹窗的形式铺满屏幕，并激活高斯模糊背景。
3. **画中画小窗（Picture-in-Picture Mini-Window）**：当处于全屏专注状态时，原本的中间对话流折叠为边缘的滑出式抽屉，或提供类似视频会议的“画中画”迷你聊天窗，确保教师在全屏演示 B+ 树时，依然可以随时打字发出“再插入一个节点 90”的微调指令，而无需频繁关闭弹窗。

六、针对底层缺陷的健壮性补充规范

为了彻底消除前述指出的项目架构不足，混合解决方案在落地时必须强制遵守以下两项底层技术规约：

1. 教师端频繁拖拽导致的“广播风暴”遏制（Throttling & Debounce Pipeline）

当教师在全球播放轴上疯狂拖动滑块时，前端绝对不允许将每一次像素变更都转换为 `player:seek` 事件发给后端。必须在前端 WebSocket 发送端架设一层 **时间窗口限制器（100ms 节流锁）**。在连续拖拽期间，仅向全班学生广播轻量级的“锁定占位指针”；当教师鼠标松开并停顿超过 150ms 后，才向 Redis Pub/Sub 发送最终的静态 `stepIndex` 状态包，使学生端一次性对齐，彻底杜绝网络卡顿。

2. 后端 MCP 进程崩溃时的前端“优雅优雅降级”机制（Graceful Degradation Overlay）

一旦底层的 `simulator-engine` 进程发生死锁或崩溃，导致前端通过 WebSocket 接收到 `{code: "SERVER_PROCESS_ERR", message: "..."} 异常时，预览画布不得卡死或白屏。渲染区必须立刻切入“静态 ASCII 兜底视图”或弹出提示：“🔥 动画模拟器正在热重启，已为您自动切换至静态执行计划树视图”。同时，后端 Runtime 独立利用进程守护（Supervisor 机制）在后台 1 秒内重启该 MCP 实例，并在就绪后静默推送 demo:complete，让前端无缝恢复动态演示，保障课堂教学的连贯性。`